

Learning/Optimizing Multivariate Least Squares

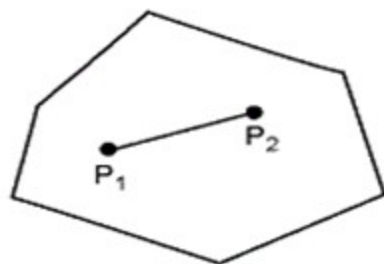
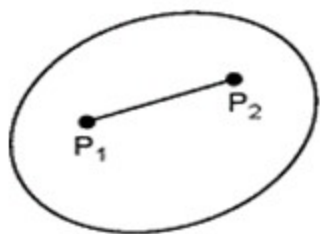
Approach 1: Gradient Descent

Unconstrained minimization problems

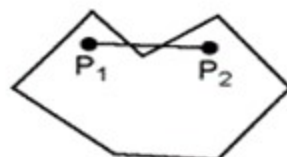
- Minimize $f(x)$
 - When f is differentiable and convex, a necessary and sufficient condition for a point x^* to be optimal is $\nabla f(x^*) = 0$
- Minimize $f(x)$ is the same as finding solution of $\nabla f(x^*) = 0$
 - Min $f(x)$: Analytically solving the optimality equation
 - $\nabla f(x^*) = 0$: Usually be solved by an iterative algorithm

Convex Set

In geometry, a subset of a Euclidean space, or more generally an affine space over the reals, is convex if, given any two points, it contains the whole line segment that joins them.



Convex set



Non-Convex set

Convex Function

Let X be a convex set in a real vector space and let $f : X \rightarrow \mathbb{R}$ be a function.

- f is called **convex** if:

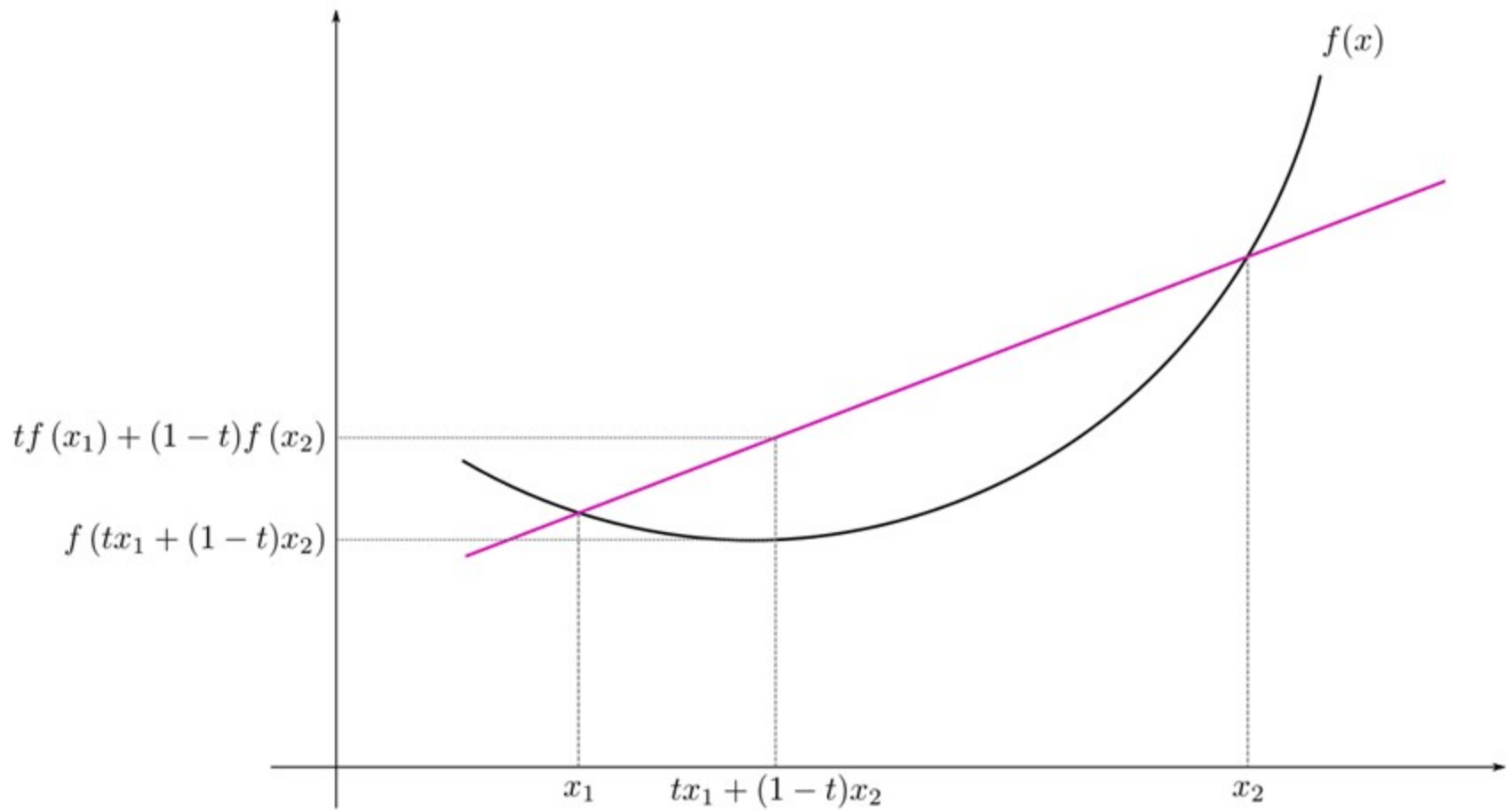
$$\forall x_1, x_2 \in X, \forall t \in [0, 1] : \quad f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

- f is called **strictly convex** if:

$$\forall x_1 \neq x_2 \in X, \forall t \in (0, 1) : \quad f(tx_1 + (1-t)x_2) < tf(x_1) + (1-t)f(x_2)$$

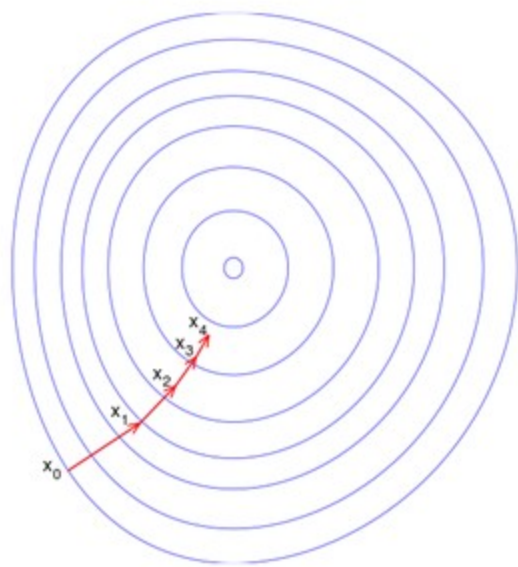
- A function f is said to be (strictly) **concave** if $-f$ is (strictly) convex.

Convex Function

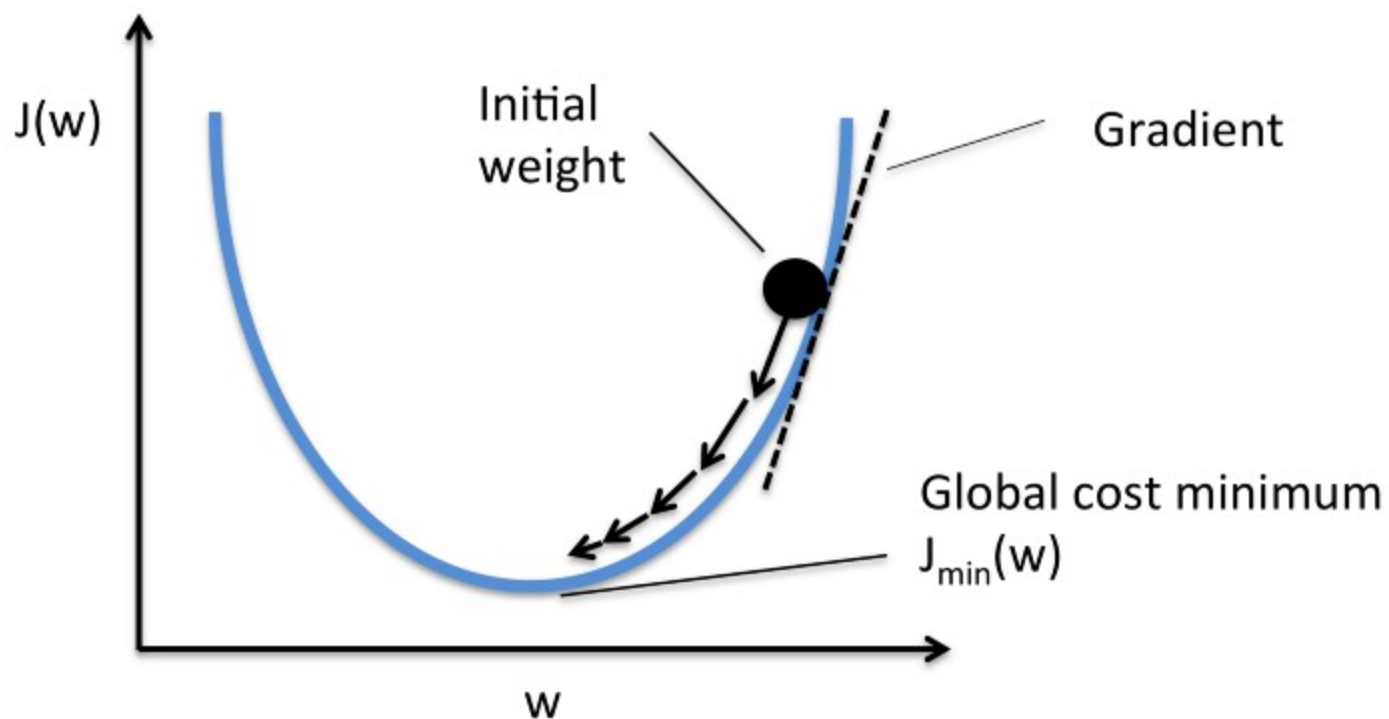


Description of Gradient Descent Method

- The idea relies on the fact that $-\nabla f(x^{(k)})$ is a descent direction
- $x^{(k+1)} = x^{(k)} - \eta^{(k)} \nabla f(x^{(k)})$ with $f(x^{(k+1)}) < f(x^{(k)})$
 - $\Delta x^{(k)}$ is the step, or search direction
 - $\eta^{(k)}$ is the step size, or step length
 - Too small $\eta^{(k)}$ will cause slow convergence
 - Too large $\eta^{(k)}$ could cause overshoot the minima and diverge



Description of Gradient Descent Method



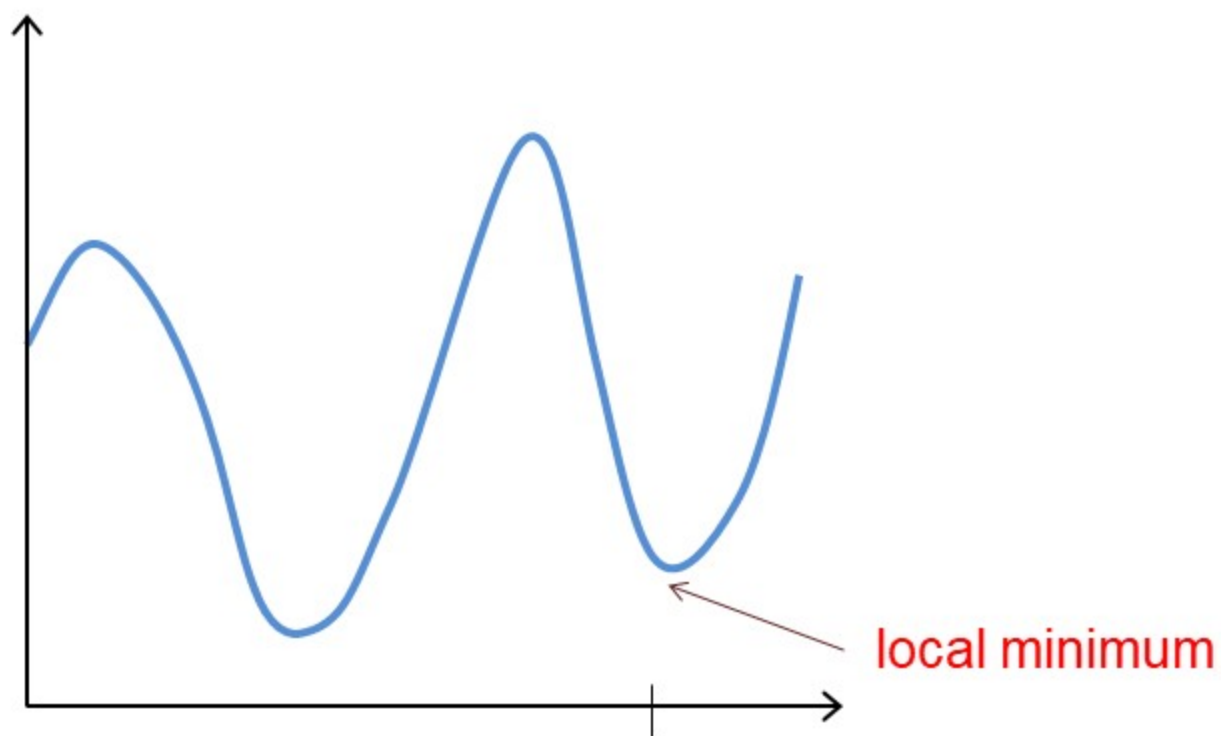
Description of Gradient Descent Method

- Algorithm (Gradient Descent Method)
 - **given** a starting point $x \in \text{dom } f$
 - **repeat**
 1. $\Delta x := -\nabla f(x)$
 2. Line search: Choose step size η via exact or backtracking line search
 3. Update $x := x + \eta \Delta x$
 - **until** stopping criterion is satisfied
- Stopping criterion usually $\|\nabla f(x)\|_2 \leq \epsilon$
- Very simple, but often very slow; rarely used in practice

Pros and Cons

- Pros
 - Can be applied to every dimension and space (even possible to infinite dimension)
 - Easy to implement
- Cons
 - Local optima problem
 - Relatively slow close to minimum
 - For non-differentiable functions, gradient methods are ill-defined

Local minimum problem

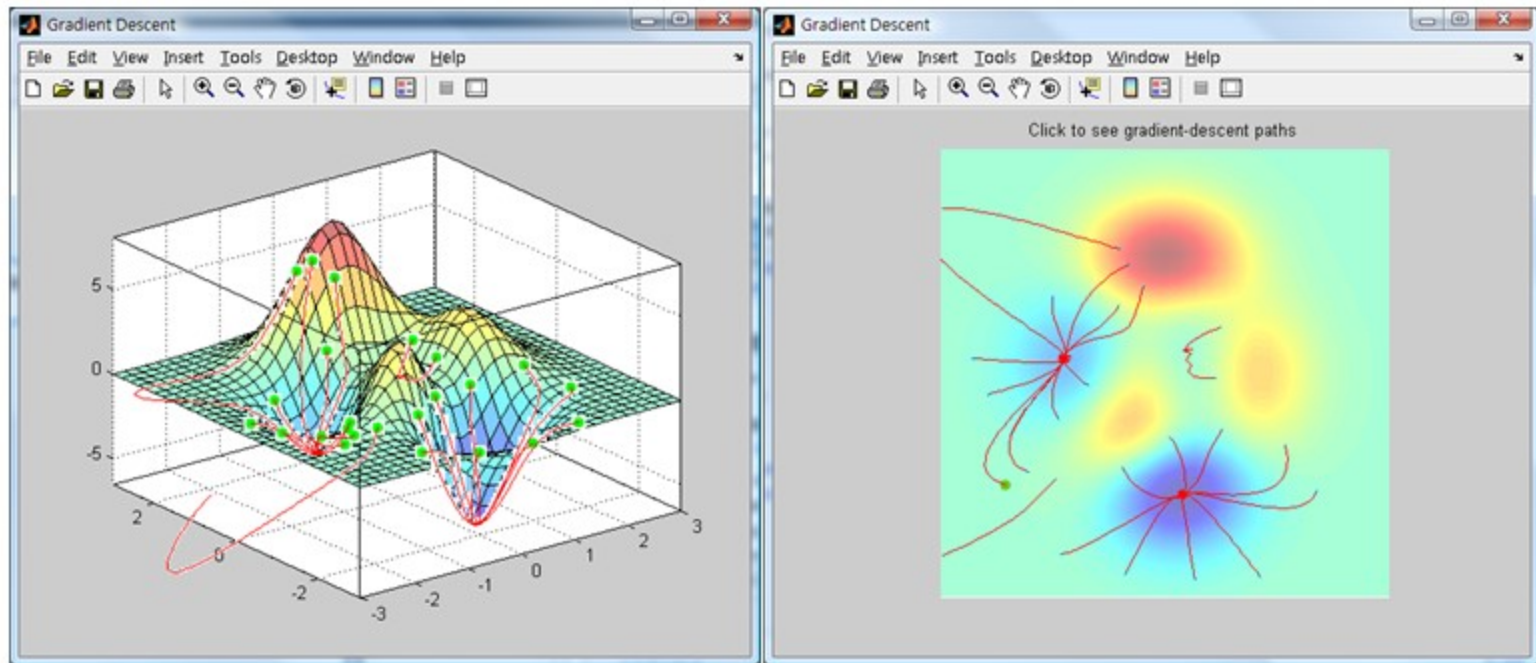


Example Code & Usage

Example Code, Usage, Questions

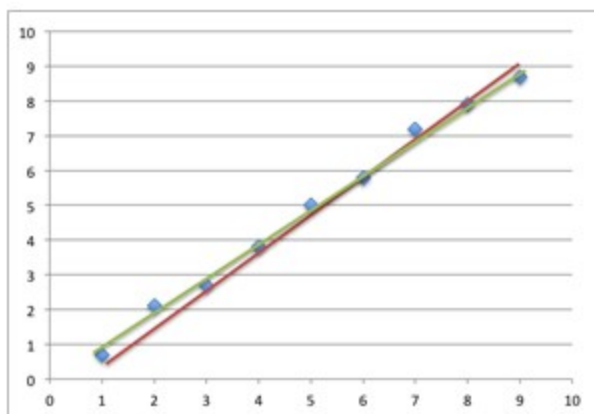
Gradient Descent Example Code

- <http://mirlab.org/jang/matlab/toolbox/machineLearning/>



Usage of Gradient Descent Method

- Linear Regression
 - Find minimum loss function to choose best hypothesis



Example of Loss function:

$$(data_{predict} - data_{observed})^2$$

Find the hypothesis (function) which minimize the loss function

Usage of Gradient Descent Method

- Neural Network
 - Back propagation
- SVM (Support Vector Machine)
- Graphical models
- Least Mean Squared Filter

...and many other applications!

Questions

- Does Gradient Descent Method always converge?
- If not, what is condition for convergence?
- How can make Gradient Descent Method faster?
- What is proper value for step size $\eta^{(k)}$

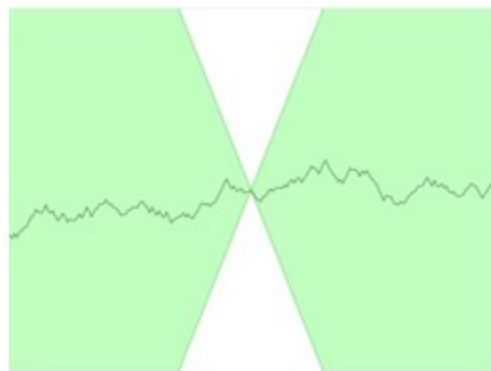
Convergence Conditions

,L-Lipschitz function

L-Lipschitz function

- Definition

- A function $f: R^n \rightarrow R$ is called *L-Lipschitz* if and only if
$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2, \forall x, y \in R^n$$
- We denote this condition by $f \in C_L$, where C_L is class of *L-Lipschitz* functions



L-Lipschitz function

- Lemma 4.1

- If $f \in C_L$, then $|f(y) - f(x) - \langle \nabla f(x), y - x \rangle| \leq \frac{L}{2} \|y - x\|^2$

- Theorem 4.2

- If $f \in C_L$ and $f^* = \min_x f(x) > -\infty$, then the gradient descent algorithm with fixed step size satisfying $\eta < \frac{2}{L}$ will converge to a stationary point

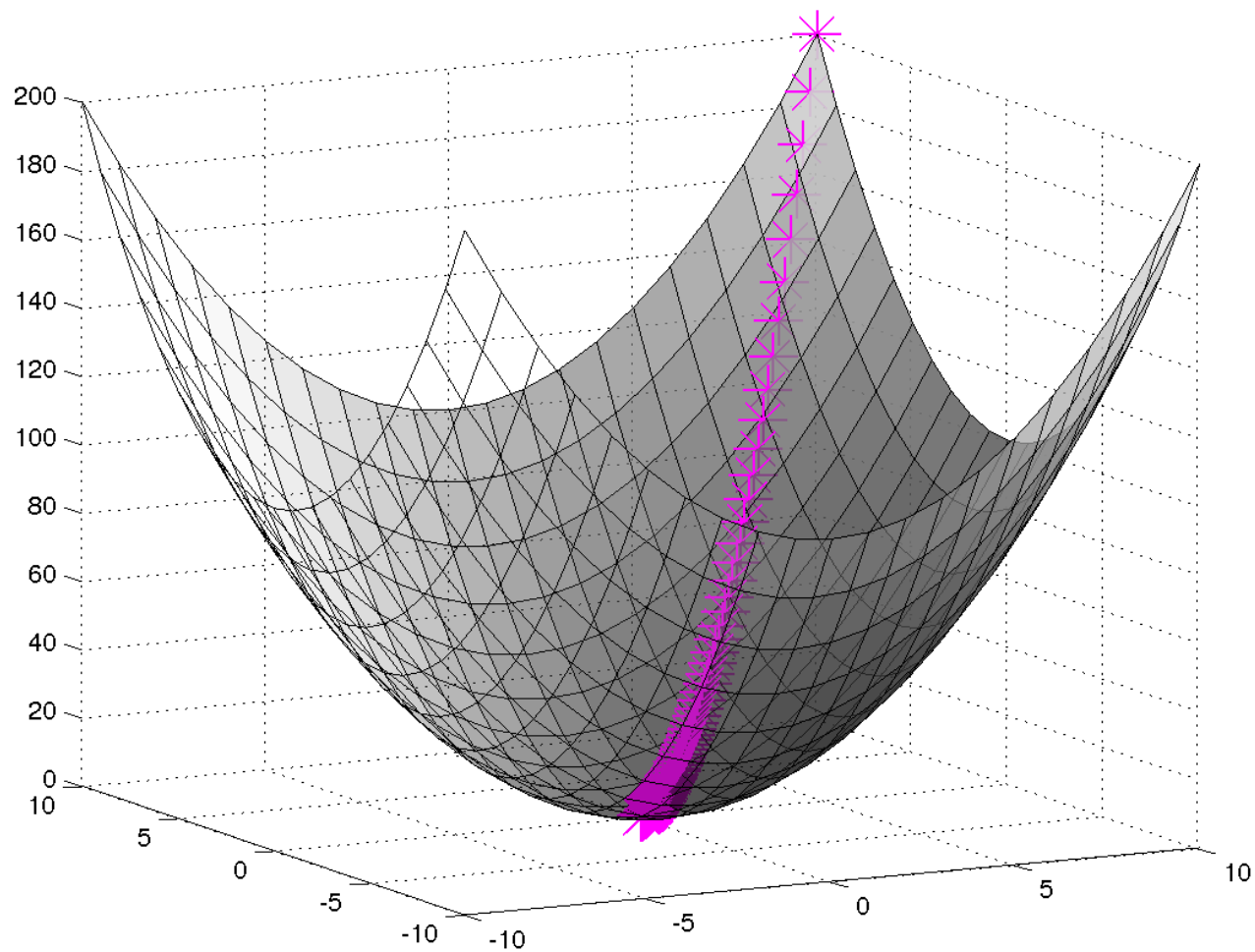
General linear regression problem

- Using our new notations for the basis function linear regression can be written as

$$y = \sum_{j=0}^n w_j \phi_j(x)$$

- Where $\phi_j(\mathbf{x})$ can be either x_j for multivariate regression or one of the non-linear basis functions we defined
- ... and $\phi_0(\mathbf{x})=1$ for the intercept term

Gradient descent



Gradient Descent for Linear Regression

Goal: minimize the following loss function:

predict with: $\hat{y}^i = \sum_j^n w_j \phi_j(\mathbf{x}^i)$

$$J_{\mathbf{X},\mathbf{y}}(\mathbf{w}) = \sum_i \left(y^i - \hat{y}^i \right)^2 = \sum_i \left(y^i - \sum_j w_j \phi_j(\mathbf{x}^i) \right)^2$$

↑
sum over n examples

↑
sum over $k+1$ basis vectors

A numeric Example

- $x^i = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad y^i = 1.5$

- $\varphi_j(x^i) = \exp((x^i - \mu^j)^T (x^i - \mu^j)) \quad \mu^j = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$

Gradient Descent for Linear Regression

Goal: minimize the following loss function: predict with: $\hat{y}^i = \sum_j^n w_j \phi_j(\mathbf{x}^i)$

$$J_{\mathbf{x},\mathbf{y}}(\mathbf{w}) = \sum_i (y^i - \hat{y}^i)^2 = \sum_i \left(y^i - \sum_j w_j \phi_j(\mathbf{x}^i) \right)^2$$

$$\begin{aligned} \frac{\partial}{\partial w_j} J(\mathbf{w}) &= \frac{\partial}{\partial w_j} \sum_i (y^i - \hat{y}^i)^2 \\ &= 2 \sum_i (y^i - \hat{y}^i) \frac{\partial}{\partial w_j} \hat{y}^i \\ &= 2 \sum_i (y^i - \hat{y}^i) \frac{\partial}{\partial w_j} \sum_j w_j \phi_j(\mathbf{x}^i) \\ &= 2 \sum_i (y^i - \hat{y}^i) \phi_j(\mathbf{x}^i) \end{aligned}$$

Gradient Descent for Linear Regression

Learning algorithm:

- Initialize weights $\mathbf{w}=\mathbf{0}$
- For $t=1, \dots$ until convergence:
 - Predict for each example $\mathbf{x}^i$ using $\mathbf{w}$:
$$\hat{y}^i = \sum_{j=0}^k w_j \phi_j(\mathbf{x}^i)$$
 - Compute gradient of loss:
$$\frac{\partial}{\partial w_j} J(\mathbf{w}) = 2 \sum_i (y^i - \hat{y}^i) \phi_j(\mathbf{x}^i)$$
 - This is a vector $\mathbf{g}$
 - Update: $\mathbf{w} = \mathbf{w} - \lambda \mathbf{g}$
 - λ is the learning rate.

Gradient Descent for Linear Regression

We can use any of the tricks

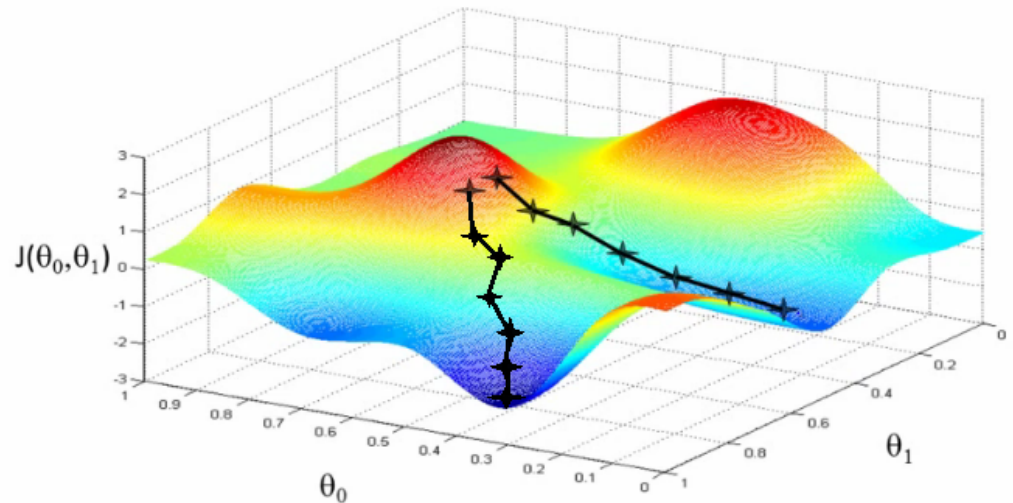
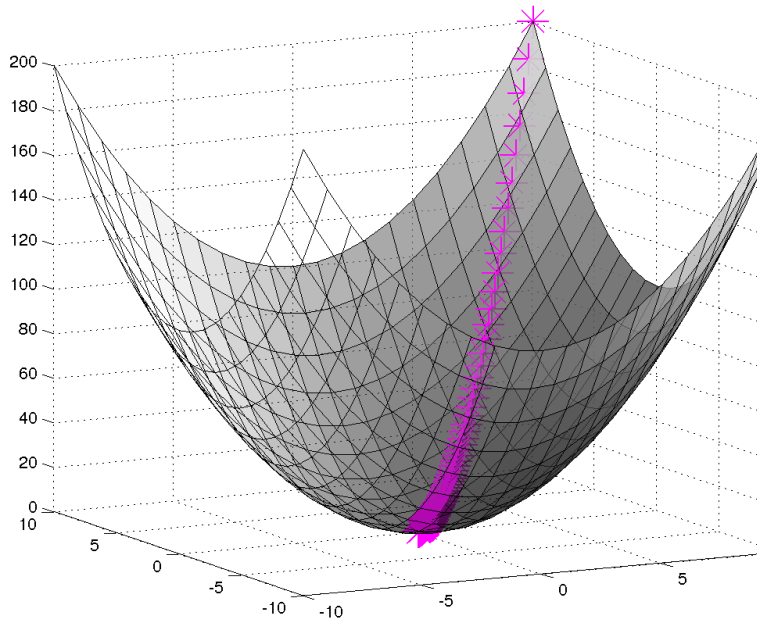
stochastic gradient descent (if the data is too big to put in
(memory

regularization

... —

Linear regression is a *convex* optimization problem

so again gradient descent will reach a *global* optimum



proof: differentiate again to get the second derivative

LMS for general linear regression problem

$$J(\mathbf{w}) = \sum_i (y^i - \mathbf{w}^T \phi(x^i))^2$$

Deriving $\mathbf{w}$ we get: $\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$

k+1 entries vector

n by k+1 matrix

n entries vector

This solution is also known as 'pseudo inverse'

Another reason to start with an objective function: you can see when two learning methods are the same!

LMS versus gradient descent

$$J(\mathbf{w}) = \sum_i (y^i - \mathbf{w}^T \phi(x^i))^2 \quad \mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

LMS solution:

- + Very simple in Matlab or something similar
- Requires matrix inverse, which is expensive for a large matrix.

Gradient descent:

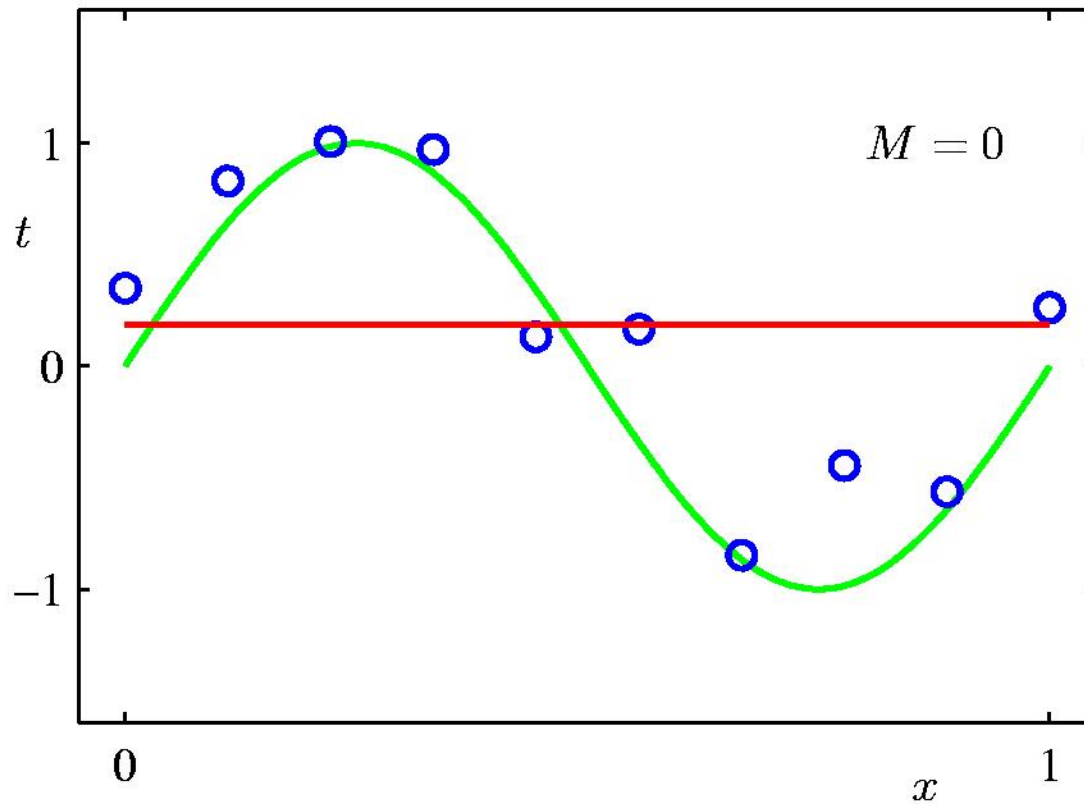
- + Fast for large matrices
- + Stochastic GD is very memory efficient
- + Easily extended to other cases
- Parameters to tweak (how to decide convergence?
what is the learning rate?)

Regression and Overfitting

An example: polynomial basis vectors on a small dataset

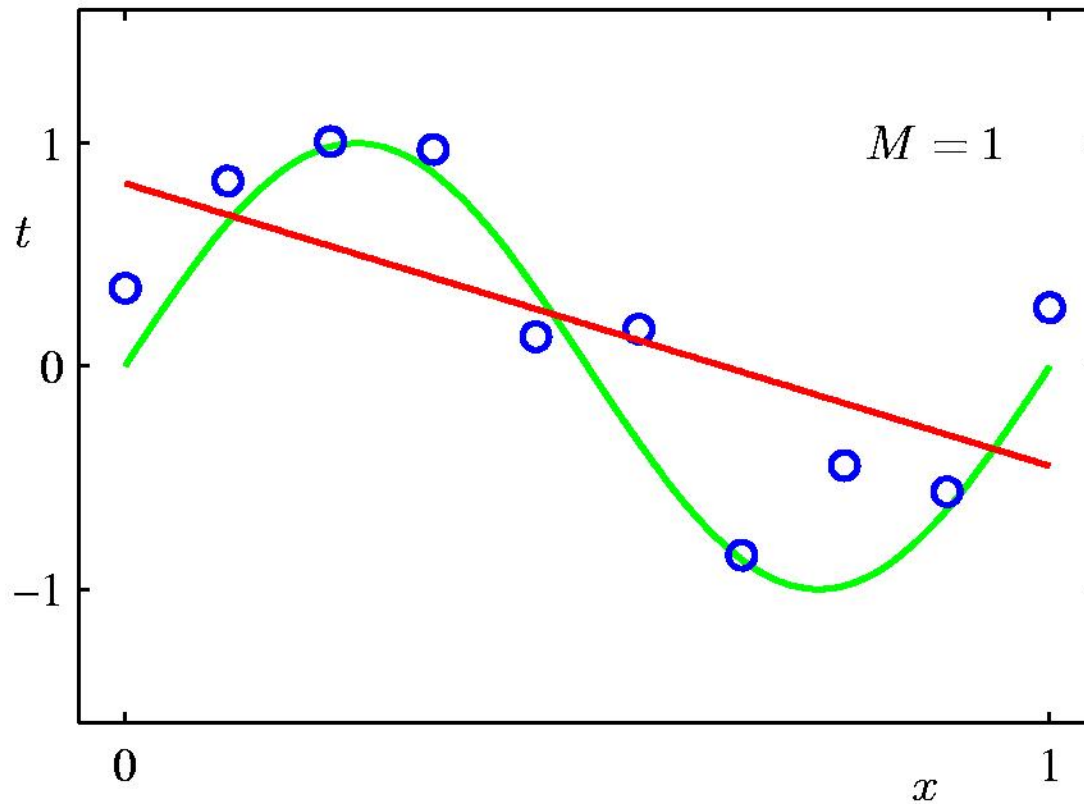
– From Bishop Ch 1

0th Order Polynomial

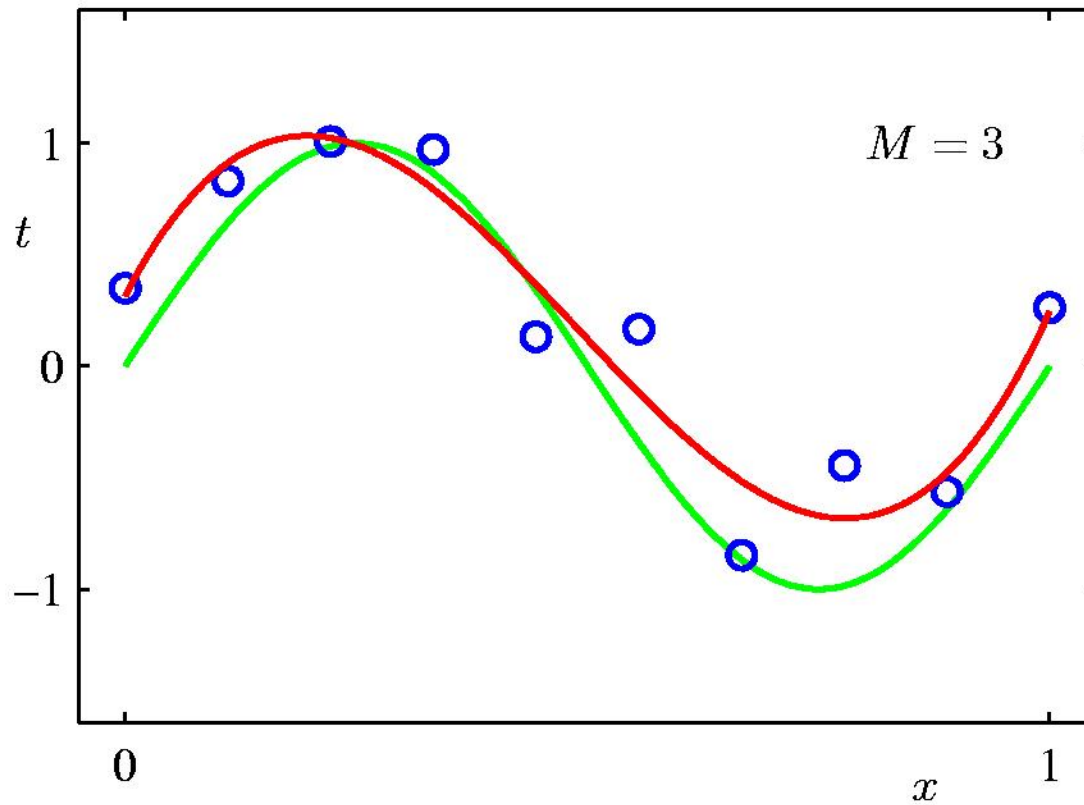


$n=10$

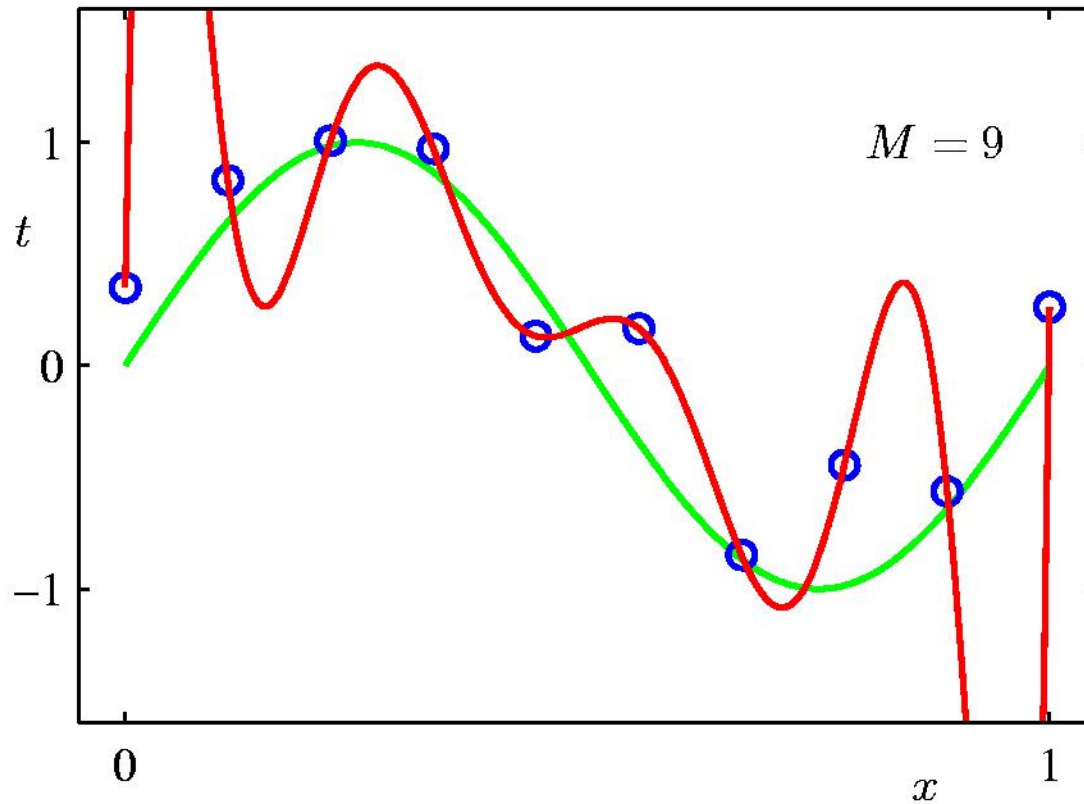
1st Order Polynomial



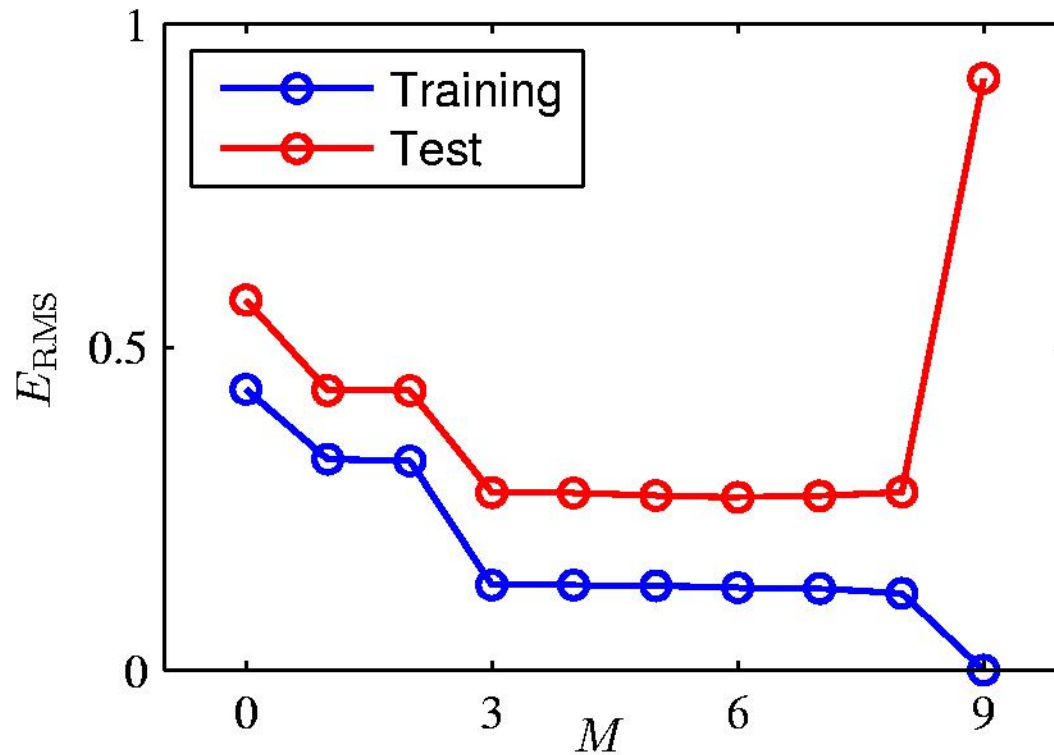
3rd Order Polynomial



9th Order Polynomial



Over-fitting



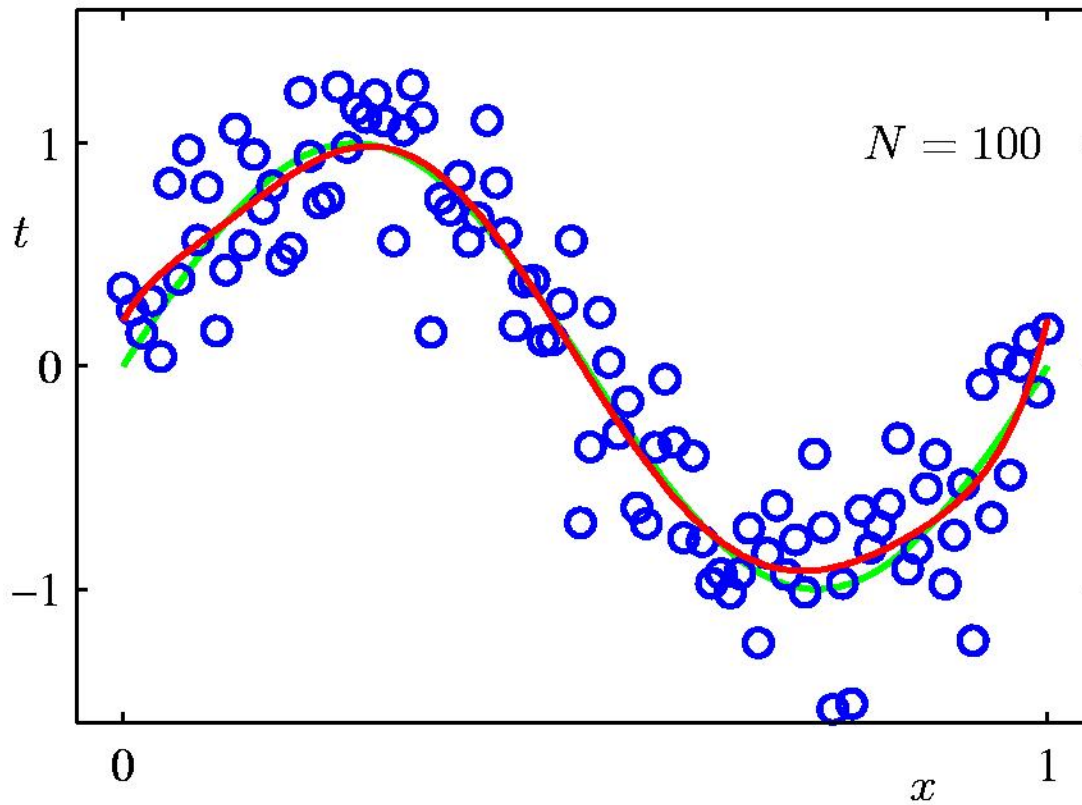
Root-Mean-Square (RMS) Error: $E_{\text{RMS}} = \sqrt{2E(\mathbf{w}^*)/N}$

Polynomial Coefficients

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

Data Set Size:

9th Order Polynomial $N = 100$



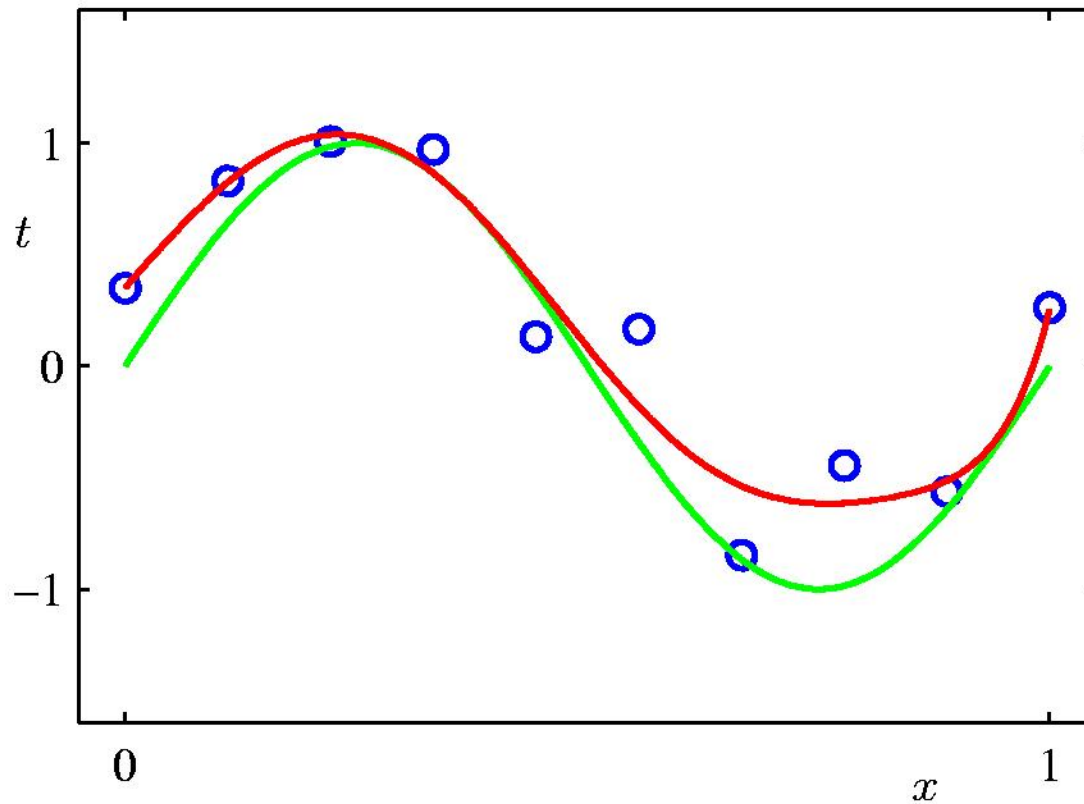
Regularization

Penalize large coefficient values

$$J_{\mathbf{x},\mathbf{y}}(\mathbf{w}) = \frac{1}{2} \sum_i \left(y^i - \sum_j w_j \phi_j(\mathbf{x}^i) \right)^2 - \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Regularization:

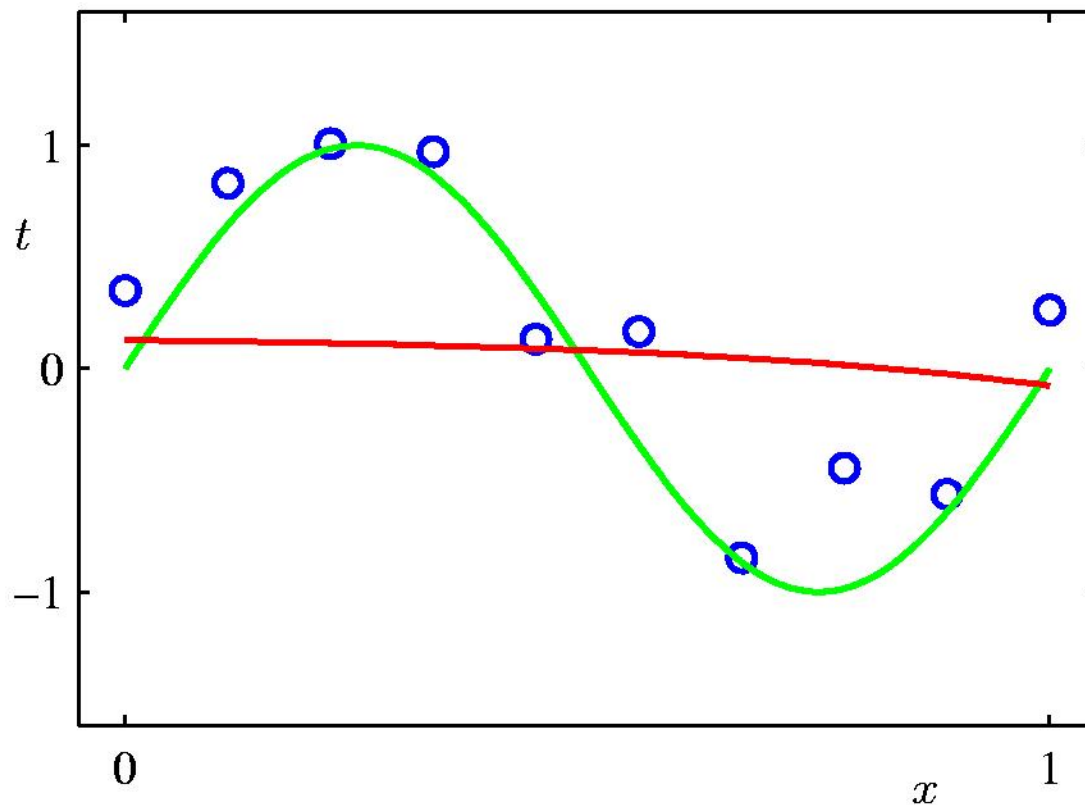
$$\ln \lambda = +.18$$



Polynomial Coefficients

	none	exp(18)	huge
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Over Regularization:



Regularized Gradient Descent for LR

Goal: minimize the following loss function: predict with: $\hat{y}^i = \sum_j^n w_j \phi_j(\mathbf{x}^i)$

$$J_{\mathbf{x},\mathbf{y}}(\mathbf{w}) = \frac{1}{2} \sum_i \left(y^i - \sum_j w_j \phi_j(\mathbf{x}^i) \right)^2 - \frac{\lambda}{2} \|\mathbf{w}\|^2$$

$$J_{\mathbf{x},\mathbf{y}}(\mathbf{w}) = \frac{1}{2} \sum_i \left(y^i - \sum_j w_j \phi_j(\mathbf{x}^i) \right)^2 - \frac{\lambda}{2} \sum_j w_j^2$$

$$\frac{\partial}{\partial w_j} J(\mathbf{w}) = \sum_i \left(y^i - \hat{y}^i \right) \phi_j(\mathbf{x}^i) - \lambda w_j$$

Probabilistic Interpretation of Least Squares

A probabilistic interpretation

Our least squares minimization solution can also be motivated by a probabilistic interpretation of the regression problem: $y = \mathbf{w}^T \phi(x) + \varepsilon$

where ε is Gaussian noise

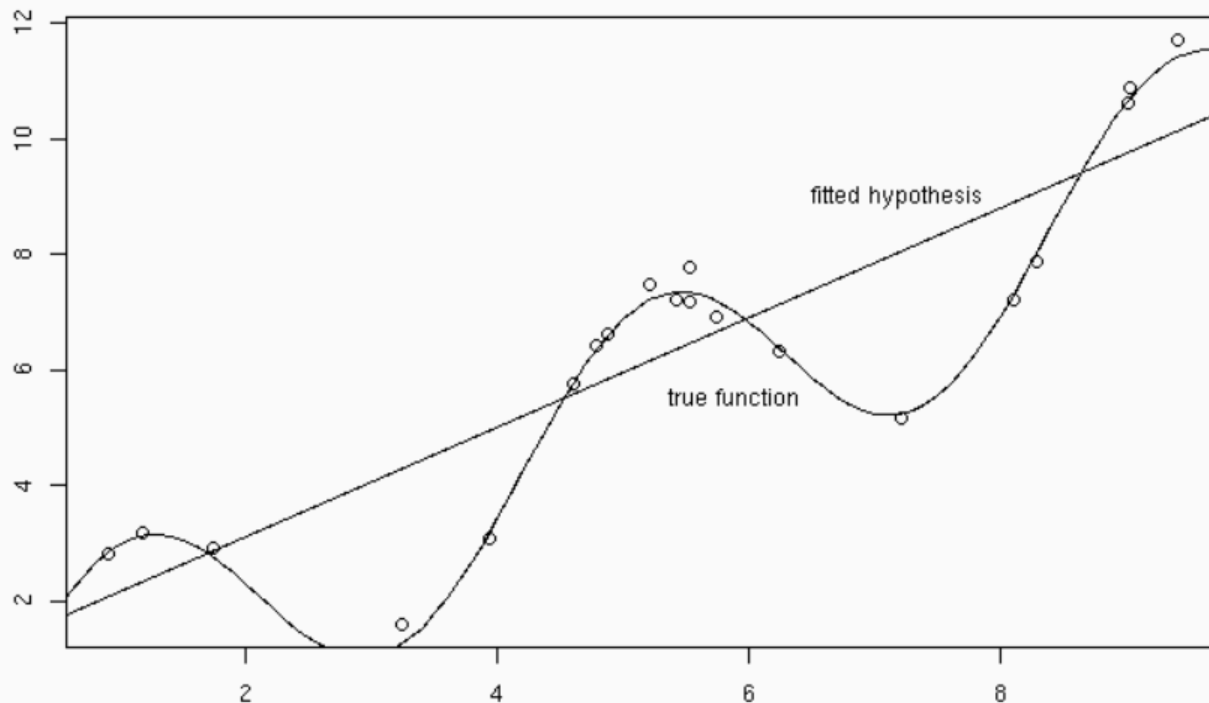
The MLE for $\mathbf{w}$ in this model is the same as the solution we derived for least squares criteria:

$$\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

Bias-Variance

Example

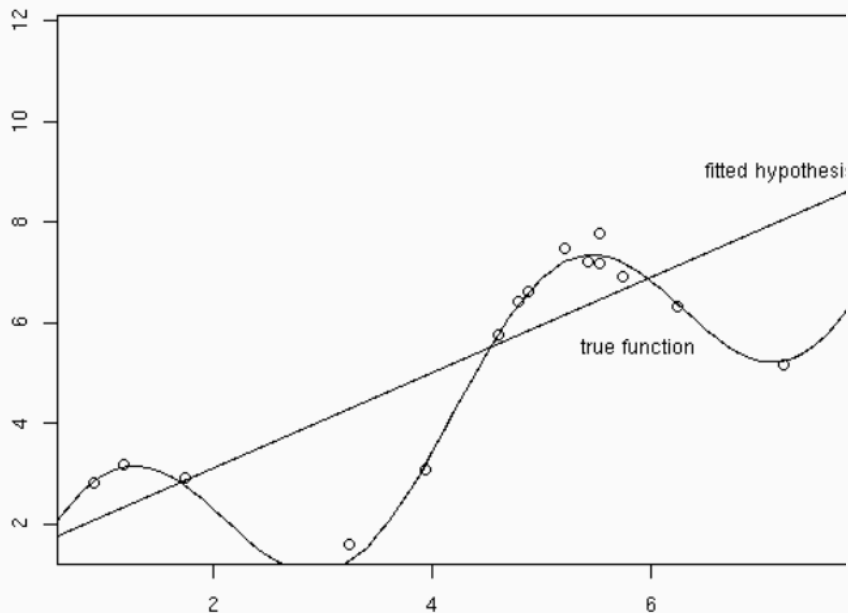
Tom Dietterich, Oregon St



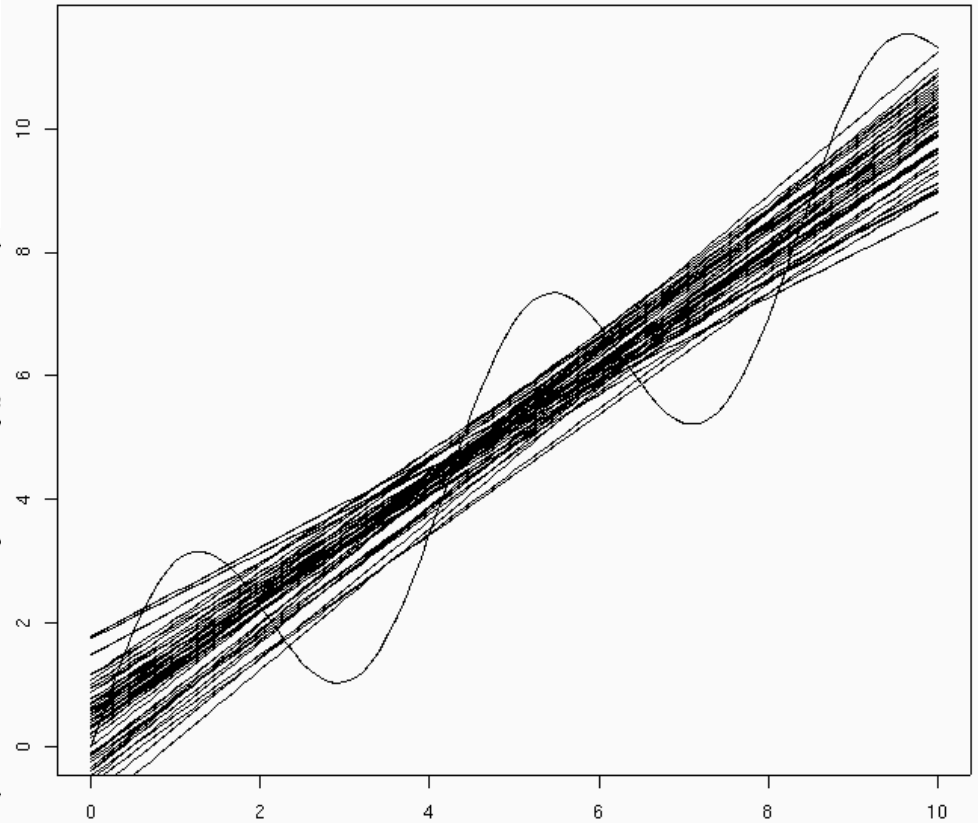
$$y = x + 2 \sin(1.5x) + N(0,0.2)$$

Example

Tom Dietterich, Oregon St



$$y = x + 2 \sin(1.5x) + N(0,0.2)$$



Same experiment, repeated:
with 50 samples of 20 points each